

SQL SYNTAX

INTRODUCTION:

Introduction to create, read, update, and delete (CRUD). These four behaviours they are also the first syntax we must understand.

BASIC SYNTAX:

```
SELECT `id`, `name` FROM `new_schema`.`users`;
```

1.Create data:

INSERT Syntax:

```
INSERT INTO `new_schema`.`users` (`id`, `name`, `age`) VALUES (4, 'Harry', 33);
```

- INSERT INTO: These two can be put together and are mainly used to specify the databasetable and columns to be added.
- VALUES: The value to be added will be set after this word, and they will need to correspond to the columns specified after INSERT INTO.

2.Read Data:

SELECT Syntax:

```
SELECT `id`, `name` FROM `new_schema`.`users`;
```

SELECT is used to retrieve data from a table

SELECT All Syntax:

```
SELECT * FROM `new_schema`.`users`;
```

Using an asterisk (*), instead of listing the column names, will fetch all of the columns in the table, and you will get the all data in table.

3.Modify Data:

UPDATE Syntax:

```
UPDATE `new_schema`.`users` SET `name` = 'Andy', `age` = 100 WHERE `id` = 2;
```

- UPDATE: Following this keyword is the name of the table to be updated.
- SET: Following are the value(s) that should be updated in the existing data. If there are multiple columns, separate them with commas.

4.Remove Data:

DELETE Syntax:

```
DELETE FROM `new_schema`.`users` WHERE `id` = 1
```

The only new keyword here is DELETE. It is connected to FROM, which is a keyword we have seen before and refers to which table to operate from.

WHERE

The WHERE clause is used to filter records in a SQL query. It tells the database which rows you want to retrieve or modify.

Syntax:

SELECT column_name

FROM table_name

WHERE condition;

Common WHERE operators

- `=` → equal to
- `>` → greater than
- `<` → less than
- `>=` → greater than or equal to
- `<=` → less than or equal to
- `<>` or `!=` → not equal to

What is JSON in SQL

If you mean JSON in SQL it stands for JavaScript Object Notation.

JSON is a format used to store structured data as key-value pairs.

Example of JSON

```
{  
  "name": "Arun",  
  "age": 25,  
  "city": "Coimbatore"  
}
```

Here:

- `"name"` → key
- `"Arun"` → value
- `"age"` → key
- `25` → value

AUXILLARY SELECT STATEMENT

In MySQL, “auxiliary SELECT statement” generally refers to a SELECT query used inside another SQL statement to help produce the final result.

1.Uniqueness:

DISTINCT

DISTINCT is used with SELECT to remove duplicate values from the result.

Syntax:

```
SELECT DISTINCT column_name  
FROM table_name;
```

2.Pagination:

LIMIT & OFFSET

LIMIT:

LIMIT specifies how many rows you want

Syntax:

```
SELECT *  
FROM employees  
LIMIT 5;
```

This returns only the **first 5 rows**

OFFSET :

OFFSET specifies **how many rows to skip** before returning results

```
SELECT *
```

```
FROM employees
```

```
LIMIT 5 OFFSET 5;
```

Skip the first **5** rows.

Return the next **5** rows.

3.Sorting:

ORDER

ORDER BY is used to sort the results of a SQL query.

You can sort data in:

- **Ascending order** → ASC
- **Descending order** → DESC

Syntax:

```
SELECT column_name
```

```
FROM table_name
```

```
ORDER BY column_name ASC;
```

4.Grouping:

GROUP BY

GROUP BY is used to group rows that have the same value in one or more columns.

It is commonly used with **aggregate functions** such as:

- COUNT() → counts rows
- SUM() → calculates total
- AVG() → calculates average
- MAX() → finds maximum
- MIN() → finds minimum

Example table

Suppose we have an employees table id
name department salary

1	Arun IT	30000
2	Priya HR	40000
3	Ravi IT	50000
4	Kumar HR	35000
5	Anu IT	45000

1.Group by with COUNT()

Suppose you want to know how many employees are in each department

Syntax:

```
SELECT department, COUNT(*)  
FROM employees  
GROUP BY department;
```

2. Group by with SUM()

To calculate the total salary for each department:

Syntax:

```
SELECT department, SUM(salary)  
FROM employees  
GROUP BY department;
```

3.Group by with AVG()

To calculate the average salary in each department.

Syntax:

```
SELECT department, AVG(salary)
FROM employees
GROUP BY department;
```

4.Group by with MAX()

Highest salary in each department.

Syntax:

```
SELECT department, MAX(salary)
FROM employees
GROUP BY department;
```

5.Group by with min()

Lowest salary in each department

Syntax:

```
SELECT department, MIN(salary)
FROM employees
GROUP BY department;
```